Program 1: Importing and Exploring Datasets

Aim

To import a dataset using Python libraries and perform basic data exploration to understand its structure, contents, and summary statistics.

Software Requirements

- Python 3.x
- Jupyter Notebook / Google Colab / VS Code
- Libraries:
    - pandas
    - numpy
    - matplotlib
    - seaborn

Theory

Data exploration is the first step in any Data Science project. Before applying machine learning algorithms or statistical analysis, we need to understand the dataset.

The **Pandas** library provides powerful functions to:

- Load datasets
- View records
- Check data types
- Find missing values
- Generate summary statistics

Algorithm

1. Import the required libraries.

2. Load the dataset using Pandas.
3. Display the first and last few records.
4. Display dataset information.
5. Check the dimensions of the dataset.
6. Display column names.
7. Generate descriptive statistics.
8. Check for missing values.
9. Find unique values in categorical columns.

Your program is **correct** and demonstrates the basic steps of **Importing and Exploring a Dataset** using Pandas. It is suitable for beginners and can be executed in **Google Colab**, **Jupyter Notebook**, or **VS Code** (provided the required libraries are installed).

Program Explanation

Step 1: Import Libraries

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

Imports the required Python libraries:

- **pandas** – Data manipulation and analysis.
- **numpy** – Numerical computations.
- **matplotlib.pyplot** – Data visualization.
- **seaborn** – Statistical data visualization.

If Using Google Colab

Before reading the file, upload it:

from google.colab import files

```
uploaded = files.upload()


import pandas as pd
df = pd.read_csv("student_data.csv")
```

Step 2: Load Dataset

```
df = pd.read_csv("student_data.csv")
```

Reads the CSV file into a Pandas DataFrame.

Note: Ensure that student_data.csv is uploaded to the current working directory (or Google Colab).

Step 3: Display Dataset

```
print(df.head())
print(df.tail())
```

- head() displays the first five records.
- tail() displays the last five records.

Step 4: Dataset Information

```
print(df.info())
```

Displays:

- Number of rows and columns
- Column names
- Data types
- Non-null values
- Memory usage

Step 5: Dataset Shape

print(df.shape)

Shows:

(Number of Rows, Number of Columns)

Step 6: Column Names

print(df.columns)

Displays all column names in the dataset.

Step 7: Descriptive Statistics

print(df.describe())

Provides statistical measures such as:

- Count
- Mean
- Standard Deviation
- Minimum
- Maximum
- Quartiles (25%, 50%, 75%)

Step 8: Missing Values

print(df.isnull().sum())

Displays the number of missing values in each column.

Step 9: Unique Values

print(df.nunique())

Displays the number of unique values in every column.

Sample Dataset (student_data.csv)

Student_ID	Name	Age	Gender	Marks	Department
101	Rahul	20	Male	85	CSE
102	Priya	21	Female	92	IT
103	Kiran	20	Male	78	CSE
104	Sneha	22	Female	88	ECE
105	Arjun	21	Male	95	IT

Expected Output (Sample)

First Five Records

	Student_ID	Name	Age	Gender	Marks	Department
0	101	Rahul	20	Male	85	CSE
1	102	Priya	21	Female	92	IT
2	103	Kiran	20	Male	78	CSE
3	104	Sneha	22	Female	88	ECE
4	105	Arjun	21	Male	95	IT

Dataset Shape

(5, 6)

Column Names

Index(['Student_ID', 'Name', 'Age', 'Gender', 'Marks', 'Department'], dtype='object')

Missing Values

Student_ID 0

Name 0

Age 0

Gender 0

Marks 0

Department 0

dtype: int64

Learning Outcomes

After completing this program, students will be able to:

1. Import essential Python libraries for data analysis.
2. Load a CSV dataset using Pandas.
3. Explore the structure of a dataset.
4. Display records from a dataset.
5. Identify missing values.
6. Obtain descriptive statistics.
7. Determine dataset dimensions and column names.
8. Count unique values in each feature.

This is an appropriate **Experiment 1: Importing and Exploring Datasets** program for a **Data Science using Python** laboratory course.

Sample Dataset (student_data.csv)

StudentID	Name	Age	Gender	Marks	Department
101	Ravi	20	Male	85	CSE
102	Priya	19	Female	90	IT

StudentID	Name	Age	Gender	Marks	Department
103	Arun	21	Male	78	ECE
104	Sneha	20	Female	88	CSE
105	Kiran	22	Male	92	IT

Expected Output

First Five Records

StudentID Name Age Gender Marks Department

101 Ravi 20 Male 85 CSE

102 Priya 19 Female 90 IT

103 Arun 21 Male 78 ECE

104 Sneha 20 Female 88 CSE

105 Kiran 22 Male 92 IT

Dataset Shape

(5, 6)

Column Names

Index(['StudentID', 'Name', 'Age', 'Gender',

'Marks', 'Department'], dtype='object')

Summary Statistics

Age

Mean = 20.4

Marks

Mean = 86.6

Maximum = 92

Minimum = 78

Missing Values

StudentID 0

Name 0

Age 0

Gender 0

Marks 0

Department 0

Explanation of Important Functions

Function	Description
pd.read_csv()	Loads a CSV dataset into a DataFrame
head()	Displays the first 5 rows
tail()	Displays the last 5 rows
info()	Displays column names, data types, and non-null counts
shape	Returns the number of rows and columns
columns	Lists all column names
describe()	Generates statistical summary for numerical columns
isnull().sum()	Counts missing values in each column
nunique()	Counts unique values in each column

Viva Questions

1. What is Pandas?
2. What is a DataFrame?
3. What is the purpose of head() and tail()?
4. How do you check the size of a dataset?
5. What does info() display?
6. Why is data exploration important?
7. How do you identify missing values in a dataset?
8. What is the difference between shape and describe()?
9. What is the use of nunique()?
10. Which library is commonly used to load CSV files?

Learning Outcomes

After completing this experiment, students will be able to:

- Import datasets using the Pandas library.
- Explore the structure and dimensions of a dataset.
- Identify data types and missing values.
- Generate summary statistics for numerical data.
- Prepare datasets for further preprocessing and analysis.

Note: If you are using **Google Colab**, upload the student_data.csv file first using the file upload option, then use pd.read_csv("student_data.csv") to load it.

Program 2: Basic Data Structures – Working with Arrays, Lists, DataFrames, and Matrices

Aim

To understand and perform operations on basic data structures such as **Lists, Arrays, DataFrames, and Matrices** using Python libraries.

Software Requirements

- Python 3.x
- Jupyter Notebook / Google Colab / VS Code
- Libraries:
    - NumPy

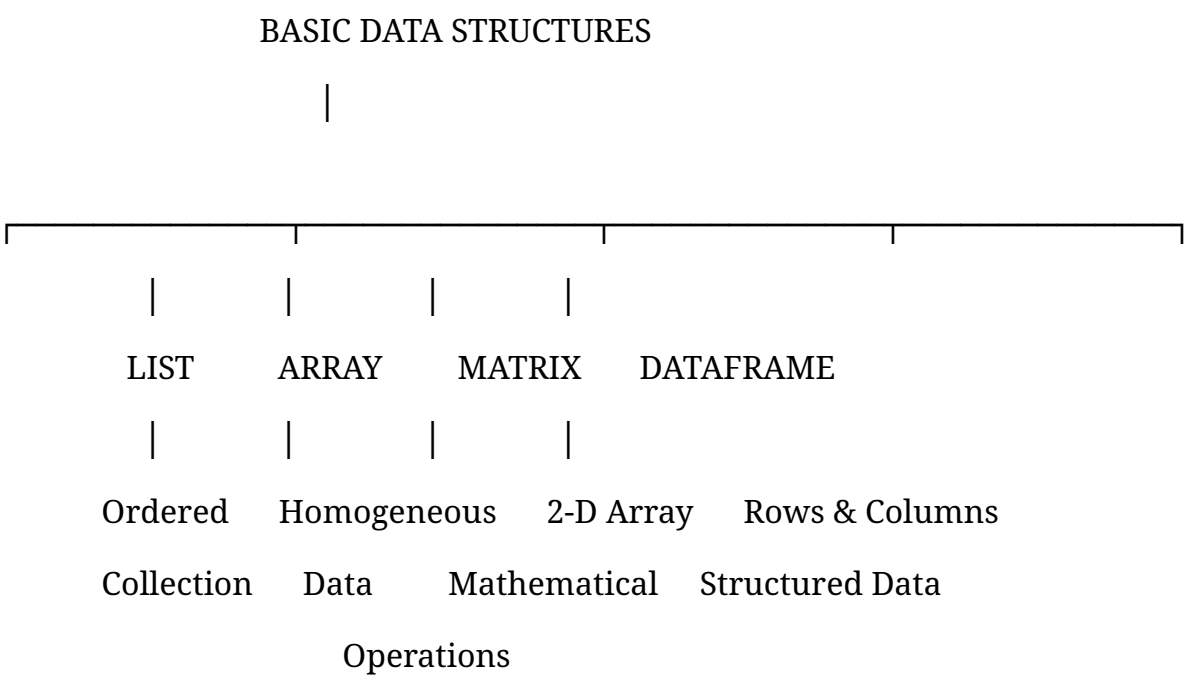
- Pandas

Theory

Data structures are fundamental components in Data Science used to organize, store, and manipulate data efficiently.

Data Structure	Library	Description
List	Python	Stores multiple items of different data types
Array	NumPy	Stores homogeneous (same type) elements efficiently
Matrix	NumPy	Two-dimensional array used for mathematical computations
DataFrame	Pandas	Tabular data structure with rows and columns

Diagram: Basic Data Structures



Algorithm

1. Import NumPy and Pandas libraries.
2. Create a Python List.
3. Create a NumPy Array.
4. Perform basic array operations.
5. Create a Matrix.
6. Perform matrix operations.
7. Create a Pandas DataFrame.
8. Display DataFrame information.

Program

```
# Import Libraries
import numpy as np
import pandas as pd


# ------------------------------
# 1. Python List
# ------------------------------


print("Python List")


students = ["Ravi", "Priya", "Arun", "Sneha", "Kiran"]


print(students)


print("First Student:", students[0])
```

```
print("Number of Students:", len(students))

# ----------------------------
# 2. NumPy Array
# ----------------------------

print("\nNumPy Array")

marks = np.array([85, 90, 78, 88, 92])

print(marks)

print("Mean Marks:", np.mean(marks))

print("Maximum Marks:", np.max(marks))

print("Minimum Marks:", np.min(marks))

# ----------------------------
# 3. Matrix
# ----------------------------

print("\nMatrix")

matrix = np.array([[10,20,30],
```

```
                [40,50,60],
                [70,80,90]])

print(matrix)

print("Transpose Matrix")

print(matrix.T)

# ------------------------------
# 4. DataFrame
# ------------------------------

print("\nDataFrame")

data = {
    'Name':['Ravi','Priya','Arun','Sneha','Kiran'],
    'Age':[20,19,21,20,22],
    'Department':['CSE','IT','ECE','CSE','IT'],
    'Marks':[85,90,78,88,92]
}

df = pd.DataFrame(data)

print(df)
```

```
print("\nShape")

print(df.shape)

print("\nColumns")

print(df.columns)

print("\nSummary")

print(df.describe())
```

Sample Output

Python List

['Ravi', 'Priya', 'Arun', 'Sneha', 'Kiran']

First Student: Ravi

Number of Students: 5

NumPy Array

[85 90 78 88 92]

Mean Marks : 86.6

Maximum Marks : 92

Minimum Marks : 78

Matrix

[[10 20 30]

[40 50 60]

[70 80 90]]

Transpose

[[10 40 70]

[20 50 80]

[30 60 90]]

DataFrame

	Name	Age	Department	Marks
0	Ravi	20	CSE	85
1	Priya	19	IT	90
2	Arun	21	ECE	78
3	Sneha	20	CSE	88
4	Kiran	22	IT	92

Summary Statistics

	Age	Marks
Count	5.0000	5.0000
Mean	20.4000	86.6000
Std	1.1402	5.4129
Min	19.0000	78.0000

25%	20.0000	85.0000
50%	20.0000	88.0000
75%	21.0000	90.0000
Max	22.0000	92.0000

Explanation of Important Functions

Function	Description
list()	Creates a Python list
np.array()	Creates a NumPy array
np.mean()	Calculates the average value
np.max()	Returns the maximum value
np.min()	Returns the minimum value
matrix.T	Computes the transpose of a matrix
pd.DataFrame()	Creates a DataFrame
shape	Returns the number of rows and columns
columns	Displays the column names
describe()	Generates descriptive statistics

Comparison of Data Structures

Feature	List	Array	Matrix	DataFrame
Library	Python	NumPy	NumPy	Pandas
Dimensions	1-D	1-D / Multi-D	2-D	2-D

Feature	List	Array	Matrix	DataFrame
Data Type	Mixed type	Same type	Same type	Mixed
Speed	Moderate	Fast	Fast	Moderate
Best Use	General storage	Numerical computation	Mathematical operations	Tabular data analysis

Applications

- **Lists:** Store names, cities, products, or any collection of items.
- **Arrays:** Numerical computations, statistical analysis, and machine learning.
- **Matrices:** Linear algebra, image processing, and scientific computing.
- **DataFrames:** Data cleaning, preprocessing, analysis, visualization, and machine learning.

Viva Questions

1. What is a data structure?
2. What is the difference between a List and a NumPy Array?
3. What is a Matrix?
4. Why are NumPy arrays faster than Python lists?
5. What is a DataFrame?
6. Which library is used to create a DataFrame?
7. How do you find the transpose of a matrix?
8. What is the purpose of describe() in Pandas?
9. What is the difference between homogeneous and heterogeneous data?

10. Why are DataFrames widely used in Data Science?

Learning Outcomes

After completing this experiment, students will be able to:

- Understand the role of Lists, Arrays, Matrices, and DataFrames in Data Science.
- Create and manipulate Python Lists and NumPy Arrays.
- Perform basic matrix operations such as transpose.
- Create and analyze tabular data using Pandas DataFrames.
- Select the appropriate data structure for different data analysis tasks.

Basic Data Structures in Data Science

Working with Arrays, Lists, Data Frames, and Matrices

Data structures are specialized formats used to organize, store, and manipulate data efficiently. In Data Science, choosing the right data structure helps perform computations quickly and effectively.

---

1. List

Definition

A **list** is an ordered collection of elements that can store different data types (integers, floats, strings, etc.). Lists are mutable, meaning their contents can be modified after creation.

Characteristics

- Ordered collection
- Mutable (can be changed)
- Allows duplicate values
- Can contain mixed data types

Example

```
# Creating a list
students = ["Alice", "Bob", "Charlie", "David"]


print(students)


# Accessing elements
print(students[0])


# Adding an element
students.append("Emma")


print(students)
```

Output

```
['Alice', 'Bob', 'Charlie', 'David']
Alice
['Alice', 'Bob', 'Charlie', 'David', 'Emma']
```

Applications

- Storing student names
- Product lists
- Employee records

---

2. Array (NumPy Array)

Definition

An **array** is a collection of elements of the same data type stored in contiguous memory. Arrays are faster and more memory-efficient than Python lists for numerical computations.

Characteristics

- Stores homogeneous data
- Faster mathematical operations
- Fixed dimensions
- Efficient memory usage

Import NumPy

```
import numpy as np
```

Example

```
import numpy as np


numbers = np.array([10, 20, 30, 40, 50])


print(numbers)


print(numbers + 5)
```

print(numbers * 2)

Output

[10 20 30 40 50]

[15 25 35 45 55]

[20 40 60 80 100]

Applications

- Scientific computing
- Machine Learning
- Image processing
- Numerical analysis

---

3. Matrix

Definition

A **matrix** is a two-dimensional array consisting of rows and columns.

Characteristics

- Two-dimensional
- Homogeneous data
- Used in linear algebra
- Supports matrix operations

Example

```
import numpy as np


matrix = np.array([[1,2,3],
                   [4,5,6],
                   [7,8,9]])
```

print(matrix)

Output

[[1 2 3]

[4 5 6]

[7 8 9]]

Matrix Addition

A = np.array([[1,2],[3,4]])

B = np.array([[5,6],[7,8]])

print(A + B)

Output

[[6 8]

[10 12]]

Matrix Multiplication

print(np.dot(A,B))

Output

[[19 22]

[43 50]]

Applications

- Artificial Intelligence
- Image Processing
- Neural Networks
- Data Transformations

---

4. Data Frame (Pandas DataFrame)

Definition

A **DataFrame** is a two-dimensional labeled data structure provided by the **Pandas** library. It is similar to a spreadsheet or SQL table.

Characteristics

- Rows and columns
- Different data types allowed
- Easy data manipulation
- Supports indexing and filtering

Import Pandas

import pandas as pd

Example

```
import pandas as pd


data = {
   "Name":["Alice","Bob","Charlie"],
   "Age":[20,21,22],
   "Marks":[85,90,88]
}


df = pd.DataFrame(data)


print(df)
```

Output

```
      Name  Age  Marks
0    Alice   20     85
1      Bob   21     90
2  Charlie   22     88
```

Selecting a Column

print(df["Marks"])

Output

0 85

1 90

2 88

Name: Marks, dtype: int64

Applications

- Data Analysis
- Machine Learning
- Business Analytics
- Data Cleaning

Comparison of Data Structures

Feature	List	Array	Matrix	Data Frame
Library	Built-in	NumPy	NumPy	Pandas
Dimensions	1D	1D/ND	2D	2D
Mixed Data Types	Yes	No	No	Yes
Mutable	Yes	Yes	Yes	Yes
Speed	Moderate	Fast	Fast	Fast
Mathematical Operations	Limited	Excellent	Excellent	Good
Labels	No	No	No	Yes

Real-Time Examples

Data Structure	Example
List	Student names in a class
Array	Daily temperature readings
Matrix	Image pixels (rows × columns)
Data Frame	Student database with Name, Roll No., Marks

Advantages

List

- Easy to use
- Flexible
- Supports mixed data types

Array

- High performance
- Efficient memory usage
- Vectorized computations

Matrix

- Ideal for mathematical operations
- Supports linear algebra
- Widely used in AI and graphics

Data Frame

- Handles structured data
- Easy filtering and grouping

- Rich data analysis capabilities

Summary

- **List:** Flexible collection of elements; suitable for general-purpose data storage.
- **Array:** Efficient numerical data structure provided by NumPy for fast computations.
- **Matrix:** Two-dimensional array mainly used for mathematical and scientific operations.
- **DataFrame:** Tabular data structure in Pandas with labeled rows and columns, ideal for data analysis and machine learning.

These four data structures form the foundation of data handling in Python-based Data Science workflows and are extensively used in Google Colab, Jupyter Notebook, and other Python development environments.